

SimCSE for Encrypted Traffic Detection and Zero-Day Attack Detection

ROTEM BAR¹ AND CHEN HAJAJ²

¹School of Software Engineering: Intelligent Systems, Afeka College of Engineering, Tel Aviv 6910717, Israel

²Data Science and Artificial Intelligence Research Center, Industrial Engineering and Management Department, Ariel University, Ariel 4076414, Israel

Corresponding author: Rotem Bar (rotem.bar@s.afeka.ac.il)

ABSTRACT Traffic detection has attracted much attention in recent years, playing an essential role in intrusion detection systems (IDS). This paper proposes a new approach for traffic detection at the packet level, inspired by natural language processing (NLP), using simple contrastive learning of sentence embeddings (SimCSE) as an embedding model. The new approach can learn the features of traffic from raw packet data. Experiments were conducted on two well-known datasets to evaluate our approach. For detecting malicious activity, our model achieved an accuracy of 99.99% on the USTC-TFC2016 dataset, whereas for detecting virtual private network (VPN) activity, our model achieved an accuracy of 99.98% on the ISCXVPN2016 dataset. Furthermore, the resulting model was found to be robust based on zero-day attack detection, which shows the model's ability to detect attacks that have not been seen before. Experiments show that our approach can effectively detect network traffic and outperforms many other state-of-the-art methods.

INDEX TERMS Network traffic, SimCSE, packet capture, Word2vec, cyber security.

I. INTRODUCTION

Traffic detection plays an essential role in intrusion detection systems (IDS). An IDS collects network traffic behavior through network devices (switches, routers, and network taps) and analyzes the difference in information sources to determine whether the network system is under malicious activity that can cause theft or damage to hardware or software and disrupt the services they provide. One of the IDS tasks involves detecting a VPN activity. A VPN extends a private network across a public network and enables users to send and receive data across public networks as if their computing devices are directly connected to a private network. Such VPN services may be used by users who aim to hide their malicious intent from network systems.

In recent years, most traffic detection studies have focused on statistical and behavioral-based approaches [1]–[4], that analyze traffic by extracting features from observational data using a set of specific features. Although the statistical-based and behavioral-based approaches have the advantage of creating new features by manipulating existing features, it faces a challenge in extracting an optimal feature representation due to the required prior knowledge of traffic properties.

The associate editor coordinating the review of this manuscript and approving it for publication was Jinming Wen.

To overcome this challenge, many recent studies have focused on packet-level techniques [5]–[8]. Traffic detection performed at the packet level is a rapidly new developing machine learning approach performed on the byte label and solves the problem of hand-designing features. Packet-level techniques for network traffic can be divided into two main approaches. One approach is to take particular first values of the byte labels while dropping the rest and feeding these byte values into a deep neural network [6], [8]–[10], which is usually based on convolutional layers utilized to find patterns. The second approach uses embedding models [5], [11], [12] inspired by the natural language processing (NLP) field, which transforms instances into a vector space on which a classification model can be performed. The embedding model can find patterns in the data and eliminate noise [13], [14] in the instances created by the encrypted payload component.

This paper presents a new method for representing network traffic using SimCSE [15] as an embedding model. We conducted several experiments for traffic detection on two well-known datasets to examine our model, with each dataset used for a different experiment. First, the USTC-TFC2016 dataset was used for malicious traffic detection and zero-day malicious traffic detection experiments containing benign and malicious instances. Second, the ISCXVPN2016 dataset used for the traffic detection experiment contains instances

that used or did not use a VPN service. Experimental results show that our method performs better than many previous state-of-the-art methods in this area.

The rest of this paper is organized as follows. Section II describes the related work. Section III describes the theoretical background of this study. Section IV describes the design and implementation of the proposed method. Section V covers the datasets and experimental results, and Section VI presents conclusions and future work.

II. RELATED WORK

A. EMBEDDING TECHNIQUES

In 2019, Ramström [12] proposed a hybrid model integrating a Word2vec model named IP2vec, performed on the byte label and features extracted from the packets, with both fed into an autoencoder model. To transform the data into vectors, the vectors were extracted from the latent space of the autoencoder. The authors examined several traffic application sources with an average AUC of 97.5%. Later, in 2020, Li *et al.* [11] proposed a term frequency-inverse document frequency (TF-IDF) weighted Word2vec model for traffic anomaly detection. The proposed approach compared to other embedding models, such as bag-of-words (BOW), doc2vec, Glove, and FastText. Two models were employed to perform classification: categorical boosting (CatBoost) and light-gradient boosting machine (LightGBM). The model results for the CSIS2010 dataset obtained an accuracy (ACC) of 99.49%, and for the WNSW-NB dataset, an ACC of 99.14%. During the same year, Dvir *et al.* [16] introduced an unsupervised model for clustering YouTube streaming based on Word2vec and a deep encoder for feature embedding. Several clustering algorithms were examined in the authors' work, such as k-means, GMMa, and AHC, and several metrics were used to evaluate the quality of the clusters. The best model yielded a clustering purity (CP) of 0.74. Similarly, Goodman *et al.* [5] presented the Packet2Vec, a model for traffic classification based on the Word2vec algorithm. This study used the DARPA-2009 dataset, and two classifiers were tested: random forest and naive base. In addition, the running time per thread was examined to evaluate the model classification performance. The best scoring model obtained an AUC of 99.6%. Roberto *et al.* [17] presented a comprehensive comparison of several embedding algorithms from the NLP domain, including BOW, CountVectorizer, Word2vec (avg), Word2vec (TF-IDF), and Doc2Vec. For the classification models, the authors used an ensemble model that contained support vector machines (SVM), random forest (RF), decision trees, and multilayer perceptron (MLP). The model that performed the best was Word2vec (TF-IDF), obtaining 99.91%, 100%, and 94.14% F1 scores on the datasets WWW2019, NGIDS-DS, and ADFA-LD, respectively. In 2021, Livermore *et al.* [18] proposed an industrial control system device classification model using network traffic. The proposed approach separates the architecture into two stages, named IP2Vec and DNP2Vec; both are based on the Word2vec algorithm. In the IP2Vec stage, the instances

were classified as ICS and non-ICS devices. In the second stage, performing only on the ICS device instances, the instances were classified into different ICS device classes using an SVM classifier. The F1 scores for the two ICS datasets were 91% and 97.4%, respectively.

B. DEEP NEURAL NETWORKS TECHNIQUES

In 2017, Wang and Zang [19] presented a traffic classification method using a convolutional neural network (CNN) by converting traffic data into images. The input for the CNN model was the first 784 byte values, which created $28 \times 28 \times 256$ images. These images were trained in a CNN-based model with the same architecture as Le-Net5. The results obtained from the ISCXVPN2016 dataset showed an ACC of 98.6%. Later, in 2019, Hwang *et al.* [8] introduced a new approach based on CNN and long short-term memory (LSTM) layers. The input for the model was built from the first 54 byte values converted to vectors using one-hot encoding. The proposed model examined four different datasets: ISCX-IDS-2012, USTC-TFC-2016, Mirai-RGU, and Mirai-CCU, with ACC values of 99.99%, 99.99%, 100%, and 99.4 - 100%, respectively. In 2020, Wang *et al.* [6] presented an architecture based on a CNN and gated recurrent unit (GRU). In this work, the first 54 byte values were used as the input for the model. Three datasets were examined to evaluate the model performance: ISCX2012, USTC-TFC-2016, and CICIDS2017, in which ACC of 99.42%, 99.94%, and 100% was obtained respectively. In 2020, Shapira and Shavitt [9] proposed a different approach. The first 1500 byte values were used for the input, while instances with smaller lengths were dropped. A histogram of size 1500 was built from the instance packet, creating a $1500 \times 1500 \times 2$ size image for each instance. The architecture examined for classification was based on the Le-Net5 architecture, and an ACC of 98.4% was obtained for the ISCXVPN2016 dataset. In 2020, Bendiab *et al.* [10] compared four models based on CNN and recurrent neural network (RNN): Resnet34, Resnet50, MobileNet, and SOINN. The input for the model was the first 784 byte values that were transformed into RGB images with a size of 1024×256 . Resnet50 shows the best results from the four architectures with an ACC of 94.5% on a dataset that Bendiab *et al.* built from several sources. In 2022, Bader *et al.* [20] proposed a hybrid model called MalDIST. MalDIST consists of three modules. The first module input contains the first 784 byte values of the byte label, and for the second module, the input was features extracted from the first 32 packets of the PCAP file from which the instance came. The third module input contains statistical features obtained from the first 32 packets from the PCAP file from which the instance came. Each module input was fed into a deep learning network with a different architecture. Next, the output from the three modules was connected to one 384-sized vector fed into an additional deep neural network to perform classification. The data set used by Bader *et al.* is an assembly of three datasets: StratosphereIPS, ISCX2016, MTA, and the results obtained have an accuracy of 98.96%.

TABLE 1. Related work summary.

Category	Author	Year	Methods	Dataset	Performance
Embedding	Ramström [12]	2019	Word2vec (avg) + autoencoder	CTU-13	AUC= 97.5%
	Li et al. [11]	2020	Word2vec (TF-IDF)	CSIS2010 WNSW-NB	ACC= 99.49% ACC= 99.14%
	Dvir et al. [16]	2020	Word2vec (avg) + autoencoder	10,000 YouTube stream	CP= 0.74%
	Goodman et al. [5]	2020	Word2vec (avg)	DARPA-2009	AUC=99.6%
	Roberto et al. [17]	2020	Word2vec (TF-IDF)	WWW2019 NGIDS-DS ADFA-LD	ACC= 99.91% ACC= 100% ACC= 94.14%
	Livermore et al. [18]	2021	Word2vec (avg)	ICS datasets	F1=97.4%
	Wang and Zang [19]	2017	CNN	ISCXVPN2016	ACC=99.99%
	Hwang et al. [8]	2019	CNN+LSTM	ISCX-IDS-2012 USTC-TFC-2016 Mirai-RGU Mirai-CCU	ACC=99.99% ACC=99.99% ACC=100% ACC=99.46%
Deep Neural Networks	Wang et al. [6]	2020	CNN+GRU	ISCX2012 USTC-TFC-2016 CICIDS2017	ACC=99.42% ACC=99.94% ACC=100%
	Shapira and Shavitt [9]	2020	CNN	ISCXVPN2016	ACC=98.4%
	Bendiab et al. [10]	2020	CNN+ RNN	Dataset built by the author	ACC=94.5%
	Bader et al. [20]	2022	CNN+GRU/LSTM	Dataset built by the author	ACC=98.96%

From Table 1, when we examine the embedding methods, we see that common to all these works is the use of Word2vec as the embedding model. Some of these studies, which compared other embedding methods, such as Glove, FastText, and doc2vec, showed that Word2vec presented the best results among the different embedding methods. When we examine studies that use deep neural networks, we see that the use of convolution layers is common to all. Although these architectures present good results, their main drawbacks are the model size and the processing power required for training and prediction.

III. KNOWLEDGE BACKGROUND

A. SimCSE EMBEDDING

Simple contrastive learning of sentence embedding (SimCSE) was first introduced by Gao *et al.* in 2021 [15]. Contrastive learning is a machine learning technique used to learn the general features of unsupervised data by training a model with similar and dissimilar instances.

SimCSE, as an unsupervised contrastive learning model [21], requires positive and negative pairs of input sentences for training. Therefore, a pair of instances are built as the negative pair, while for the positive pair, the model takes the same negative pair and performs dropout

to add noise to construct the positive pair, which functions as augmentation. One of the challenges is that the learned vectors occupy a narrow cone in the vector space, severely limiting their expressiveness. To overcome this challenge, SimCSE contrastive learning can be expressed by the formulas (1) and (2), where the first term keeps positive instances similar and the second term pushes negative pairs apart:

$$\ell_{\text{align}} \triangleq \mathbb{E}_{(x, x^+) \sim P_{\text{pos}}} \|f(x) - f(x^+)\|^2 \quad (1)$$

$$\ell_{\text{uniform}} \triangleq \log \mathbb{E}_{x, y \stackrel{i.i.d.}{\sim} P_{\text{data}}} e^{-2\|f(x) - f(y)\|^2} \quad (2)$$

where P_{data} denotes the data distribution. Therefore, for two instances embedded with different dropout masks ($h_i^{z_i}, h_i^{z'_i}$), the training objective of SimCSE is as shown in formula (3):

$$\ell_i = -\log \frac{e^{\text{sim}(h_i^{z_i}, h_i^{z'_i})/\tau}}{\sum_{j=1}^N e^{\text{sim}(h_i^{z_i}, h_j^{z'_j})/\tau}} \quad (3)$$

where τ is a temperature hyperparameter, $\text{sim}(h_1, h_2)$ is the cosine similarity $\frac{h_1 \cdot h_2}{|h_1| \cdot |h_2|}$, and N is the mini-batch.

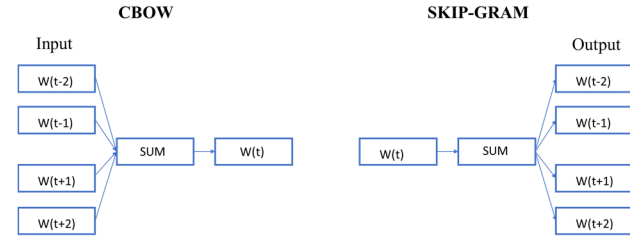


FIGURE 1. Word2vec model architecture.

Algorithm 1 Simple Average Sentence Embedding**Input:** Word embeddings $v_w: w \in V$ a set of sentences S **Output** Sentence embeddings $\{v_s: s \in S\}$

- 1: **for all** sentence s in S do
- 2: $v_s \leftarrow \frac{1}{|s|} \sum_{w \in s} word2vec(w)$
- 3: **end for**

B. WORD2VEC EMBEDDING

A large body of work that uses embedding methods has focused on the Word2vec model [5], [11], [12]. This choice was based on the fact that Word2vec presented the best capabilities among the examined embedding models. Therefore, we will compare our approach to three Word2vec variants used as a baseline.

Word2vec was first presented in 2013 by Mikolov et al [22]. Word2vec is a word-embedding model that uses two-layer neural networks trained to reconstruct the linguistic contexts of words. Word2vec takes a large corpus of text as its input and produces a vector space, with each unique word in the corpus being assigned a corresponding vector in the space. The vectors are learned using the cosine similarity function, which indicates the semantic similarity between words represented by those vectors. The model includes continuous bag-of-words (CBOW) and skip-gram training models. As shown in Fig. 1, CBOW predicts the probability of the headword by context and Skip-Gram predicts the probability of the context by headword.

Next, we describe three different sentence embedding algorithms: simple average, term frequency-inverse document frequency (TF-IDF), and smooth inverse frequency (SIF). Each algorithm uses the word-embedding model, Word2vec, to transform the instances into a uniform vector space.

Algorithm 1 is the simplest sentence embedding method that can be implemented. Extract the word vector of each word in the sentence and average all the word embeddings in the sentence.

Algorithm 2 uses the same basis as Algorithm 1, where each word vector is multiplied by the TF-IDF weights [23]. TF represents the word frequency across all sentences, and IDF represents how common or rare a word is in a sentence. The closer the IDF is to 0, the more common the word. This metric can be calculated by taking the total number of

Algorithm 2 TF-IDF Sentence Embedding**Input:** Word embeddings $v_w: w \in V$ a set of sentences S , and estimated probabilities $\{p(w): w \in V\}$ of the words.**Output:** Sentence embeddings $\{v_s: s \in S\}$

- 1: **for all** sentence s in S do
- 2: $tf_w \leftarrow \frac{p(w)}{|s|}$
- 3: $idf_w \leftarrow \log\left(\frac{|S|}{1+p(w)}\right)$
- 4: $v_s \leftarrow \frac{1}{|s|} \sum_{w \in s} tf_w \cdot idf_w word2vec(w)$
- 5: **end for**

Algorithm 3 SIF Sentence Embedding**Input:** Word embeddings $\{v_w: w \in V\}$ a set of sentences S , parameter a , estimated probabilities $\{p(w): w \in V\}$ of the words.**Output:** Sentence embeddings $\{v_s: s \in S\}$

- 1: **for all** sentence s in S do
- 2: $v_s \leftarrow \frac{1}{|s|} \sum_{w \in s} \frac{a}{a+p(w)} word2vec(w)$
- 3: **end for**
- 4: From matrix X whose columns $\{v_s: s \in S\}$, and let u be its first singular vector
- 5: **for all** sentence s in S do
- 6: $v_s \leftarrow v_w - uu^T v_s$
- 7: **end for**

instances, dividing it by the number of instances containing a word, and calculating the logarithm.

Algorithm 3 combines the concepts implemented in Algorithms 1 and 2 and performs singular vector dropping. In 2016, Arora *et al.* introduced the smooth inverse frequency (SIF) [24] algorithm, where each word vector is multiplied by one weight represented by the following: $\frac{a}{a+p(w)}$ where $p(w)$ is the probability of a word appearing in all instances, and a is a hyperparameter that increases or decreases the weight. The first singular vector is dropped in the second stage (line 6), containing common words with low meaning.

C. SVM AND RANDOM FOREST CLASSIFIERS

SVM [25], as shown in Fig. 2, is a nonprobability binary classifier model. The SVM classifier finds a decision boundary in the feature space that separates data objects into two classes during training. The optimization problem involves finding the decision boundary with the maximum separation (margin) between the two classes. The margin of a hyperplane is the distance between parallel equidistant hyperplanes on either side of the hyperplane, such that the gap is void of data objects. The optimization during training finds a hyperplane that has the maximum margin. The SVM then uses that hyperplane to predict the class of a new data object once it is presented with its feature vector. In our study, we used the radial basis function (RBF) kernel, which has the advantage of flexible boundary separation.

Random forest [26] (RF) is an ensemble learner that generates many tree-based classifiers and aggregates their results. Each classifier in the ensemble model is a binary

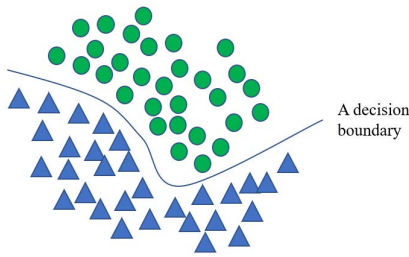


FIGURE 2. SVM classifier with RBF kernel.

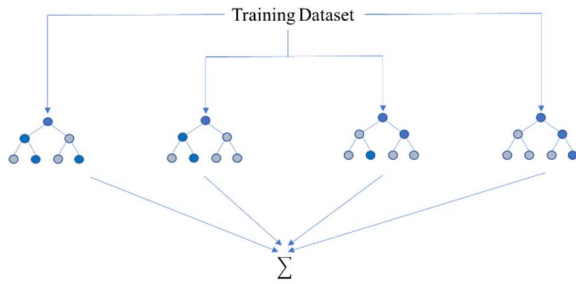


FIGURE 3. Random Forest classifier architecture.

decision tree constructed by repeatedly splitting the data into child nodes, starting with the root node containing the entire learning sample. The random forest model creates multiple classification trees, each trained on a bootstrap sample of the original training data, and searches across a randomly selected subset of input variables to determine the split. As shown in Fig. 3, each tree in a random forest will cast a vote for some input x ; then, the output of the classifier is determined by the majority voting of the trees.

IV. METHODOLOGY

This section describes the methodology used in this study. First, we briefly introduce packet capture, also known as a PCAP file, and its components. Packet capture is an application programming interface (API) that captures live networks collected from networking equipment, such as routers or switches, and works by creating copies of some or all packets passing through a given point in the network. The packets contain two components: payload and headers. The payload, which is typically encrypted, consists of the actual data being transferred. This could be audio, video, or email. The headers contain all technical information such as IP addresses and protocols. Thus, packets are duplicate copies of the actual packets traversing the network.

In our study, we used two datasets containing PCAP files: USTC-TFC2016 and ISCXVPN2016. A complete description of the datasets is provided in detail in section V. In the first stage, the byte label is extracted from the PCAP files, with each extracted byte-label instance of a different size. In the second stage, the byte label instances

are fed into an embedding model, transforming the traffic byte label instances into vectors in a size of 100. When we project the use of embedding models from the NLP field to the network traffic field, we refer to the byte label of each packet as a sentence and every paired-byte value, which is one hexadecimal value, as a word. Thus, certain hexadecimal values represent specific fields in the packet. The last step is to train a classification model on the vectors received in the previous step. The classification model classifies malicious/benign instances in the USTC-TFC2016 dataset and non-VPN/VPN instances in the ISCXVPN2016 dataset.

The described methodology integrates two types of machine learning models: embedding and classification. Hence, for the embedding models in the experimental phase, we used Word2vec (avg), Word2vec (TF-IDF), Word2vec (SIF), and SimCSE. For the classification models, we used random forest and SVM. Our proposed approach using SimCSE can be found in the GitHub repository [27].

V. EXPERIMENTAL RESULTS

A. DATA DESCRIPTION

As shown in Tables 2 and 3, we used the following two datasets in our study: the USTC-TFC2016 dataset containing benign/malicious traffic instances and ISCXVPN2016 containing non-VPN/VPN traffic instances. As in Wang et al [6], we performed a balanced sampling from the data. In contrast to preserving the original distribution, balancing the data has several advantages. The main advantage of working with balanced data is eliminating the possibility of the model being biased towards a specific class and decreasing overfitting problems. As described in the next section, for Experiments I and II, the data were split in an 80-20% ratio, with 80 percent of the instances used for training and 20 percent for testing.

B. EVALUATION METHOD

Three metrics are commonly used to evaluate balanced binary classification: accuracy (ACC), true positive rate (TPR), and false-positive rate (FPR). Their definitions are as follows.

$$ACC = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

$$TPR = \frac{TP}{TP + FN} \quad (5)$$

$$FPR = \frac{FP}{TP + TN} \quad (6)$$

True positive (TP) is the number of instances classified correctly as malicious/VPN. True negative (TN) is the number of instances classified correctly as benign/non-VPN. False-positive (FP) is the number of instances classified incorrectly as malicious/VPN, and false-negative (FN) is the number of instances classified incorrectly as benign/non-VPN. As such, the accuracy measures the efficiency of the model, the TPR measures the portion of instances that are correctly classified, and the FPR indicates the false-alarm rate (i.e., the cases when the model classified VPN/malicious instances as non-VPN/benign).

TABLE 2. USTC-TFC2016 dataset description.

Category	Benign			Category	Malicious		
	All	Train	Test		All	Train	Test
Weibo	2.6M	4.8K	1.2K	Neris	497K	4.8K	1.2K
SMB	925K	4.8K	1.2K	Shifu	497K	4.8K	1.2K
FTP	360K	4.8K	1.2K	Cridex	461K	4.8K	1.2K
MySQL	200K	4.8K	1.2K	Virut	437K	4.8K	1.2K
WorldOfWarcraft	140K	4.8K	1.2K	Nsis-ay	351K	4.8K	1.2K
Gmail	25K	4.8K	1.2K	Geodo	213K	4.8K	1.2K
BitTorrent	15K	4.8K	1.2K	Htbot	169K	4.8K	1.2K
Outlook	15K	4.8K	1.2K	Zeus	86K	4.8K	1.2K
Skype	12K	4.8K	1.2K	Miuref	81K	4.8K	1.2K
Facetime	6K	4.8K	1.2K	Tinba	21K	4.8K	1.2K
Total	2.8M	48K	12K		4.3M	48K	12K

TABLE 3. ISCXVPN2016 dataset description.

Category	Non-VPN			Category	VPN		
	All	Train	Test		All	Train	Test
Chat	82K	8K	2K		50K	8K	2K
Email	28K	8K	2K		251K	8K	2K
File Transfer	210K	8K	2K		13K	8K	2K
Streaming	1.1M	8K	2K		479K	8K	2K
VoIP	5.1M	8K	2K		753K	8K	2K
Total	6.6M	40K	10K		1.8M	40K	10K

TABLE 4. Malicious activity detection results on the USTC-TFC2016 dataset.

Embedding Model	SVM			Random Forest		
	ACC	TPR	FPR	ACC	TPR	FPR
Word2vec (avg)	98.63%	98.64%	1.38%	98.22%	97.03%	0.59%
Word2vec (TF-IDF)	95.90%	96.46%	4.66%	99.11%	98.63%	0.41%
Word2vec (SIF)	98.41%	97.64%	0.82%	97.41%	95.65%	0.83%
SimCSE	99.99%	99.99%	4.6x10⁻⁶%	99.99%	99.99%	9.8x10⁻⁶%

C. EXPERIMENT RESULT

To evaluate the performance of our approach and examine the effectiveness of the embedding models in network traffic detection tasks, we performed three experiments conducted on a CPU: Intel Xeon CPU E5-2683 2.4 GHz with 64GB RAM, and GPU: 2x Nvidia GeForce RTX 2080Ti, with Windows 10 operating system. The experiments address two tasks in the cybersecurity field: traffic detection and zero-day attack detection. The traffic detection task suits the condition

where a trained model has learned different types of network traffic data, benign and malicious, so that the test traffic data come from the same type of network traffic data as seen before. In our case, the model aims to classify test instances as malicious or benign in the USTC-TFC2016 dataset, and as non-VPN or VPN in the ISCXVPN2016 dataset. Note that the field of malicious activity is not static; new attacks are frequently designed and executed. The task of detecting new attacks is commonly referred to as zero-day attack detection.

TABLE 5. VPN activity detection results on the ISCVPN2016 dataset.

Embedding Model	SVM			Random Forest		
	ACC	TPR	FPR	ACC	TPR	FPR
Word2vec (avg)	96.20%	95.76%	3.37%	96.31%	96.34%	3.73%
Word2vec (TF-IDF)	92.63%	89.58%	4.32%	96.28%	96.13%	3.57%
Word2vec (SIF)	97.41%	97.27%	2.45%	96.21%	96.43%	3.34%
SimCSE	99.96%	99.94%	0.02%	99.98%	99.99%	0.03%

In the zero-day attack-detection task [28]–[31], the model aims to detect a new attack that has not been previously seen before. This task serves when the model does not have enough data to learn the new attack until the defender has collected sufficient instances of this attack.

1) EXPERIMENT I

Malicious activity detection employed on the USTC-TFC2016. The purpose of this experiment is to examine SimCSE's ability to represent traffic instances for detecting malicious network traffic compared to the three variants of the Word2vec model: Word2vec (avg), Word2vec (TF-IDF), and Word2vec (SIF). Additionally, we examine the outcome of integrating the specified embedding models with random forest and SVM as classifiers. This experiment was performed using five-fold cross-validation, and the results shown in Table 4 are the mean values.

2) EXPERIMENT II

VPN activity detection employed on the ISCVPN2016 dataset. This experiment examines SimCSE's ability to represent traffic instances for detecting VPN network traffic. Similar to Experiment I, SimCSE was compared with the three variants of the Word2vec model by integrating the two classifiers. This experiment was performed using five-fold cross-validation, and the results shown in Table 5 are the mean values.

3) EXPERIMENT III

Zero-day attack detection employed on the USTC-TFC2016 dataset. This experiment examines ten different attacks, each containing a different family of malicious instances. The training process for each attack is accomplished on all other families, that is, on nine malicious families and ten benign families. Each time, malicious instances of the family that were not part of the training set comprised the test set. Thus, an effective classifier will classify the entire test set as malicious, even though it has never seen this type of attack. This experiment demonstrates the robustness of the model to new attacks that have not been seen in the training stage.

From Table 4, the SimCSE results were the best, with an ACC of 99.99% for both classifiers. The second-best

result was Word2vec (TF-IDF) with random forest when this integration presented an ACC result of 99.11%. In contrast, Word2vec (TF-IDF) with SVM presented the lowest results, with an ACC of 95.90% and an FPR of 4.66%. Comparing Word2vec (SIF) and Word2vec (avg), we did not observe a significant difference between the two embedding models, and there was no significant difference between the classifiers' results. Examining the FPR results in Table 4, excluding Word2vec (TF-IDF) with SVM, we can see that Word2vec (avg) with SVM obtained an FPR of 1.38%, which is a significantly higher value than the other results obtained, showing that this model classified more benign instances as malicious.

From Table 5, SimCSE results were the best for this dataset as well, with a slight preference for the random forest classifier, where a 99.98% ACC was obtained. The second-best result was Word2vec (TF-IDF), with Random Forest receiving an ACC of 97.41%. In contrast, the lowest result was obtained by Word2vec (TF-IDF) with SVM, where ACC was 92.63% and FPR was 4.32%. When comparing the two classifiers' integration with SimCSE, in Tables 4 and 5, no significant difference in results can be seen.

From Table 6, SimCSE with SVM shows superior ability over all other models in the zero-day attack detection task, with an average TPR of 99.99%. The second-best result was obtained by Word2vec (TF-IDF) with Random Forest, in which an average TPR of 97.85% was obtained. In the SimCSE test with random forest, we observed a significant decrease in performance; therefore, this model was unable to detect any attack with a TPR of 100%. In comparing the two classifiers, we see a better preference for the SVM classifier over the random forest, except for the integration with the Word2vec (TF-IDF) embedding model, as shown in Tables 4 and 5, and in Li *et al.*'s [11] work, where this model shows promising results with a tree-based classifier.

D. COMPARISON WITH OTHER PUBLISHED METHODS

This section compares the results shown in the previous section with the results obtained in previous works. For this purpose, we compare both embedding methods and deep neural network methods. We will apply the same datasets examined in the experimental section for the embedding

TABLE 6. Zero-day attack detection TPR results on the USTC-TFC2016 dataset.

Embedding Model	Classifier	Cridex	Geodo	Htbot	Miuref	Neris	Nsis-ay	Shifu	Tinba	Virut	Zeus	TPR (avg)
Word2vec (avg)	RF	100%	99.83%	77.65%	100%	97.52%	49.65%	99.62%	100%	97.12%	98.67%	92%
	SVM	100%	99.93%	83.63%	99.63%	99.25%	77.17%	99.93%	100%	98.7%	99.31%	95.42%
Word2vec (TF-IDF)	RF	100%	100%	93.38%	100%	96.22%	87.48%	99.93%	100%	99.10%	99.73%	97.58%
	SVM	100%	99.80%	90.88%	99.95%	95.87%	88.72%	99.88%	100%	95.62%	99.20%	96.99%
Word2vec (SIF)	RF	98.48%	97.85%	67.22%	98.52%	97.28%	82.45%	93.97%	93.78%	96.68%	77.57%	90.38%
	SVM	98.11%	98.76%	79.76%	94.85%	99.7%	89.81%	98.86%	95.73%	98.81%	99.11%	94.75%
<i>SimCSE</i>	RF	99.87%	86.52%	94.58%	99.83%	99.78%	99.18%	94.28%	99.88%	99.88%	82.88%	95.67%
	SVM	100%	100%	99.9%	100%	99.9%	100%	100%	100%	100%	100%	99.99%

TABLE 7. Performance comparison with other published methods for USTC-TFC2016.

Method	ACC	TPR	FPR
Word2vec(avg) + Random Forest [5]	98.22%	97.03%	0.59%
Word2vec(TF-IDF) + LGBM [11]	99.23%	98.87%	0.41%
CNN+ LSTM [8]	99.99%	99.99%	$1.1 \times 10^{-7}\%$
CNN +GRU [6]	99.94%	99.99%	0.99%
ours	99.99%	99.99%	$4.6 \times 10^{-6}\%$

TABLE 8. Performance comparison with other published methods for ISCXVPN2016.

Method	ACC	TPR	FPR
Word2vec(avg) + Random Forest [5]	96.31%	96.34%	3.73%
Word2vec(TF-IDF) + LGBM [11]	96.39%	96.18%	3.38%
CNN [19]	99.99%	99.99%	N/A
ours	99.98%	99.99%	0.03%

methods when using the same architecture in the published papers. Therefore, Word2vec (avg) was employed with the random forest classifier, as presented in Goodman et al [5], and Word2vec (TF-IDF) was employed with the LGBM classifier, as shown in Li et al [11]. In addition to these methods, we present the results obtained in [8], [19], [20], as shown in the original papers.

From Tables 7 and 8, SimCSE presents the best results among the embedding methods [5], [11]. Comparing SimCSE with other neural network methods, the methods [8], [19] showed negligibly better results than our method. These methods are CNN-based. The drawbacks of CNN-based models are the processing time and power required for training. The CNN layers contain a significantly high amount of weight for training. Therefore, the same task can be addressed using a simpler model as in our approach.

VI. DISCUSSION AND CONCLUSION

This study proposed a new approach for representing network traffic based on the SimCSE embedding model, which leverages the dropout technique to create instances with augmentation. Section II reviews the related literature that addresses network traffic detection tasks using embedding and deep neural network methods. We showed that recent studies using embedding methods mainly used Word2vec-based models; therefore, SimCSE was compared to three variants of the Word2vec model in the experimental section. The detection results of our model showed 99.99% ACC capabilities in the USTC-TFC2016 dataset and 99.98% ACC in the ISCXVPN2016 dataset. In addition, we examined the ability of the model to detect new attacks using zero-day attack detection. Our model showed high robustness capabilities when integrating with an SVM classifier with an average attack detection ability of 99.99 %. Additionally, a comparison with other published methods showed that our approach outperformed many other state-of-the-art methods. In the future, we intend to test our approach on additional datasets with different encryption protocols, which may affect the effectiveness of embedding methods.

ACKNOWLEDGMENT

The authors would like to thank Amit Dvir for fruitful discussions and suggestions during the research process. In addition, thanks are sent to Adi Lichy for his help in accessing the data used in their evaluations.

REFERENCES

- [1] O. Barut, Y. Luo, T. Zhang, W. Li, and P. Li, "NetML: A challenge for network traffic analytics," 2020, *arXiv:2004.13006*.
- [2] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–35, Jul. 2021.
- [3] M. Abuthawabeh and K. Mahmoud, "Enhanced Android malware detection and family classification, using conversation-level network traffic features," *Int. Arab J. Inf. Technol.*, vol. 17, no. 4, pp. 607–614, Jul. 2020, doi: [10.34028/iajit/17/4A/4](https://doi.org/10.34028/iajit/17/4A/4).
- [4] M. Zaman, T. Siddiqui, M. R. Amin, and M. S. Hossain, "Malware detection in Android by network traffic analysis," in *Proc. Int. Conf. Netw. Syst. Secur. (NSysS)*, Jan. 2015, pp. 1–5, doi: [10.1109/NSysS.2015.7043530](https://doi.org/10.1109/NSysS.2015.7043530).
- [5] E. L. Goodman, C. Zimmerman, and C. Hudson, "Packet2Vec: Utilizing Word2Vec for feature extraction in packet data," 2020, *arXiv:2004.14477*.

- [6] B. Wang, Y. Su, M. Zhang, and J. Nie, "A deep hierarchical network for packet-level malicious traffic detection," *IEEE Access*, vol. 8, pp. 201728–201740, 2020, doi: [10.1109/ACCESS.2020.3035967](https://doi.org/10.1109/ACCESS.2020.3035967).
- [7] M. Lotfollahi, R. S. H. Zade, M. J. Siavoshani, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," 2017, *arXiv:1709.02656*.
- [8] R.-H. Hwang, M.-C. Peng, V.-L. Nguyen, and Y.-L. Chang, "An LSTM-based deep learning approach for classifying malicious traffic at the packet level," *Appl. Sci.*, vol. 9, no. 16, p. 3414, Aug. 2019, doi: [10.3390/app9163414](https://doi.org/10.3390/app9163414).
- [9] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 2, pp. 1218–1232, Jun. 2021, doi: [10.1109/TNSM.2021.3071441](https://doi.org/10.1109/TNSM.2021.3071441).
- [10] G. Bendiab, S. Shialeles, A. Alruban, and N. Kolokotronis, "IoT malware network traffic classification using visual representation and deep learning," in *Proc. 6th IEEE Conf. Netw. Softwarization (NetSoft)*, Jun. 2020, pp. 444–449, doi: [10.1109/NetSoft48620.2020.9165381](https://doi.org/10.1109/NetSoft48620.2020.9165381).
- [11] J. Li, H. Zhang, and Z. Wei, "The weighted Word2vec paragraph vectors for anomaly detection over HTTP traffic," *IEEE Access*, vol. 8, pp. 141787–141798, 2020, doi: [10.1109/ACCESS.2020.3013849](https://doi.org/10.1109/ACCESS.2020.3013849).
- [12] K. Ramström, *Botnet Detection on Flow Data Using the Reconstruction Error From Autoencoders Trained on Word2Vec Network Embeddings*. Accessed: Sep. 2019. [Online]. Available: <http://www.teknat.uu.se/student>
- [13] A. Mnih and K. Kavukcuoglu, "Learning word embeddings efficiently with noise-contrastive estimation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 26, 2013, pp. 1–9. Accessed: Dec. 2013.
- [14] K. A. Nguyen, S. S. I. Walde, and N. T. Vu, *Neural-Based Noise Filtering From Word Embeddings*. Accessed: Jan. 2020. [Online]. Available: <https://github.com/nguyenkh/NeuralDenoising>
- [15] T. Gao, X. Yao, and D. Chen, "SimCSE: Simple contrastive learning of sentence embeddings," 2021, *arXiv:2104.08821*. Accessed: Apr. 2021.
- [16] A. Dvir, A. K. Marnerides, R. Dubin, N. Golan, and C. Hajaj, "Encrypted video traffic clustering demystified," *Comput. Secur.*, vol. 96, Sep. 2020, Art. no. 101917, doi: [10.1016/j.cose.2020.101917](https://doi.org/10.1016/j.cose.2020.101917).
- [17] R. Corizzo, E. Zdravetski, M. Russell, A. Vagliano, and N. Japkowicz, "Feature extraction based on word embedding models for intrusion detection in network traffic," *J. Surveill., Secur. Saf.*, vol. 1, no. 2, pp. 140–150, 2020, doi: [10.20517/JSSS.2020.15](https://doi.org/10.20517/JSSS.2020.15).
- [18] I. Chakraborty, B. M. Kelley, and B. Gallagher, "Industrial control system device classification using network traffic features and neural network embeddings," *Array*, vol. 12, Dec. 2021, Art. no. 100081, doi: [10.1016/j.array.2021.100081](https://doi.org/10.1016/j.array.2021.100081).
- [19] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw.*, Apr. 2017, pp. 712–717, doi: [10.1109/ICOIN.2017.7899588](https://doi.org/10.1109/ICOIN.2017.7899588).
- [20] O. Bader, A. Lichy, C. Hajaj, R. Dubin, and A. Dvir, "MalDIST: From encrypted traffic classification to malware traffic detection and classification," in *Proc. IEEE 19th Annu. Consum. Commun. Netw. Conf. (CCNC)*, Feb. 2022, pp. 527–533, doi: [10.1109/CCNC49033.2022.9700625](https://doi.org/10.1109/CCNC49033.2022.9700625).
- [21] T. Chen, L. Hong, Y. Shi, and Y. Sun, "Joint text embedding for personalized content-based recommendation," 2017, *arXiv:1706.01084*.
- [22] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," 2013, *arXiv:1301.3781*.
- [23] L. Havriliant and V. Kreinovich, "A simple probabilistic explanation of term frequency-inverse document frequency (TF-IDF) heuristic (and variations motivated by this explanation)," *Int. J. Gen. Syst.*, vol. 46, no. 1, pp. 27–36, 2014. [Online]. Available: https://scholarworks.utep.edu/cs_techrep/849
- [24] S. Arora, Y. Liang, and T. Ma, "A simple but tough-to-beat baseline for sentence embeddings," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–16.
- [25] T. Evgeniou and M. Pontil, "Support vector machines: Theory and applications," in *Proc. Adv. Course Artif. Intell.*, in Lecture Notes in Computer Science: Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics, vol. 2049, 2001, pp. 249–257, doi: [10.1007/3-540-44673-7_12](https://doi.org/10.1007/3-540-44673-7_12).
- [26] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [27] Rotembaruch/Simple-Constructive-for-Packet-Embedding: An Innovative Method for Finding Malicious Activity Based on NLP Models. Accessed: Feb. 25, 2022. [Online]. Available: <https://github.com/rotembaruch/Simple-Constructive-for-Packet-Embedding>
- [28] M. Sarhan, S. Layeghy, M. Gallagher, and M. Portmann, "From zero-shot machine learning to zero-day attack detection," 2021, *arXiv:2109.14868*.
- [29] R. Tang, Z. Yang, Z. Li, W. Meng, H. Wang, Q. Li, and Y. Sun, "ZeroWall: Detecting zero-day web attacks through encoder-decoder recurrent neural networks," in *Proc. IEEE Conf. Comput. Commun.*, Jul. 2020, pp. 2479–2488. [Online]. Available: http://A.Example.COM/URL_1/URL_2/JSP_NAME_A.jsp
- [30] N. Sameera and M. Shashi, "Deep transductive transfer learning framework for zero-day attack detection," *ICT Exp.*, vol. 6, no. 4, pp. 361–367, Dec. 2020, doi: [10.1016/j.ict.2020.03.003](https://doi.org/10.1016/j.ict.2020.03.003).
- [31] H. Hindy, R. Atkinson, C. Tachtatzis, J.-N. Colin, E. Bayne, and X. Bellekens, "Utilising deep learning techniques for effective zero-day attack detection," *Electronics*, vol. 9, no. 10, p. 1684, Oct. 2020.
- [32] R.-H. Hwang, M.-C. Peng, V.-L. Nguyen, and Y.-L. Chang, "An LSTM-based deep learning approach for classifying malicious traffic at the packet level," *Appl. Sci.*, vol. 9, no. 16, p. 3414, Aug. 2019, doi: [10.3390/app9163414](https://doi.org/10.3390/app9163414).



ROTEM BAR received the B.Sc. degree in mechanical engineering from the Ben Gurion University of the Negev, Beer Sheva, Israel, in 2018. He is currently pursuing the M.Sc. degree in intelligent systems with the Afeka College of Engineering, Tel Aviv, Israel. His main research interests include machine learning and cyberspace security.



CHEN HAJAJ received the B.Sc. degree in computer engineering, the M.Sc. degree in electrical engineering, and the Ph.D. degree in computer science from Bar-Ilan University. From 2016 to 2018, he was a Postdoctoral Fellow at Vanderbilt University. He is currently a Faculty Member with the Department of Industrial Engineering and Management, the Head of the Data Science and Artificial Intelligence Research Center, and a member of the Ariel Cyber Innovation Center. His

research interests include machine learning, game theory, and cybersecurity. Specifically, the focuses of his work are on how to detect and robustify the weak-spots of AI methods (adversarial artificial intelligence) and personalized and preventative medicine using data-science-based methods.

...